DESARROLLO WEB CLIENTE - 2.º DAW

UD 02: Javascript y React

REACT

Material de estudio completo: componentes, props y JSX

Curso 2026-2027

Elaborado por: José Fernández González

Versión 1.3 · octubre de 2026

CC BY-NC-ND 4.0

https://github.com/jstechg/material-docente/tree/main/2DAW_Desarrollo_Web_Cliente

Índice de estudio

1. Qué es React y por qué trabajar con componentes
2. Estructura de un proyecto React con Vite
3. Componentes funcionales
4. JSX: reglas fundamentales
5. Expresiones y renderizado condicional
6. Listas, map() y key
7. Props: datos de un componente padre a un hijo
8. Desestructuración, valores por defecto y spread de props
9. children y composición de componentes
10. Jerarquía de componentes y flujo de datos
11. CSS, clases y recursos externos
12. Componentes de clase, estado y Hooks: contexto
13. Depuración y buenas prácticas
14. Ejemplo integrado original
15. Resumen comparativo para el examen

CÓMO ESTUDIARLO
Cada apartado se organiza en CONCEPTO, EXPLICACIÓN / QUÉ RECORDAR y EJEMPLO. Lee primero la idea clave, ejecuta el ejemplo en un proyecto Vite y modifica después algún dato para comprobar que entiendes el comportamiento.

1. Qué es React y por qué trabajar con componentes

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
React	Biblioteca de JavaScript orientada a construir interfaces de usuario a partir de componentes reutilizables. En esta unidad se utiliza para introducir el desarrollo basado en componentes.	function Saludo(){ return Hola ; }
Interfaz declarativa	Se describe qué debe mostrarse para unos datos determinados. React se encarga de actualizar la interfaz cuando cambian los datos que gobiernan el componente.	const conectado=true; return {conectado ? "En línea" : "Desconectado"} ;
Componente	Unidad reutilizable de interfaz que encapsula una parte de la presentación y, cuando procede, su lógica asociada. En React moderno se trabajará principalmente con funciones.	<CabeceraCurso />
Composición	Una interfaz se construye combinando componentes pequeños. Un componente puede contener otros componentes.	<Panel><Aviso /><Resumen /></Panel>
Renderizar	Es producir la representación de la interfaz a partir de los componentes y sus datos. React actualiza la parte necesaria cuando cambia la información.	createRoot(...).render(<App />);
React frente al DOM directo	Con JavaScript tradicional se modifican nodos del DOM de forma imperativa. Con React se expresa la interfaz en componentes y JSX; React coordina las actualizaciones.	React: return {mensaje} ;

2. Estructura de un proyecto React con Vite

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
index.html	Contiene el elemento raíz donde se montará la aplicación y carga el módulo principal generado para el proyecto.	
src/main.jsx	Es el punto de entrada habitual: obtiene el elemento raíz y renderiza el componente principal.	createRoot(document.getElementById("root")).render(<App />);
App.jsx	Componente raíz de la aplicación de aula. Desde él pueden importarse y componerse otros componentes.	export default function App(){ return <main>...</main>; }
Importar componentes	Un componente definido en otro fichero se importa antes de utilizarlo en JSX.	import Cabecera from "./components/Cabecera.jsx";
Exportación por defecto	Permite que un fichero exponga su componente principal con export default.	export default TarjetaEstado;
StrictMode	En desarrollo ayuda a detectar determinados problemas y prácticas no recomendadas. No modifica la sintaxis de los componentes.	<StrictMode><App /></StrictMode>
Servidor de desarrollo	Vite inicia un servidor local con recarga durante el desarrollo.	npm run dev

RECUERDA
Los nombres concretos de los ficheros pueden organizarse de distintas formas. En el curso se mantendrá una estructura clara y coherente para que cada componente sea fácil de localizar.

3. Componentes funcionales

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Función componente	Una función componente devuelve elementos React descritos normalmente con JSX. No debe confundirse con una función JavaScript cualquiera: se usa como etiqueta dentro de otra interfaz.	function Estado(){ return Preparado ; }
Nombre con mayúscula	Los nombres de componentes personalizados comienzan con mayúscula. Así JSX los distingue de etiquetas HTML como div o p.	function TarjetaModulo(){ ... }
Uso como etiqueta	Una vez definido o importado, el componente se inserta en JSX con sintaxis de etiqueta.	<TarjetaModulo />
Un único nodo raíz	El valor retornado debe tener una única raíz lógica. Puede ser una etiqueta contenedora o un Fragment.	return <> UD02 React </>;
Reutilización	El mismo componente puede utilizarse varias veces con datos distintos mediante props.	<Indicador texto="Compilación" />
Componente puro	Con las mismas props, un componente de presentación debería producir el mismo resultado y no modificar los datos recibidos.	function Etiqueta({texto}){ return {texto}; }

4. JSX: reglas fundamentales

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
JSX	Sintaxis que permite describir elementos de interfaz con una forma parecida a HTML dentro de JavaScript. Las herramientas del proyecto la transforman antes de ejecutarla en el navegador.	const titulo= Panel de estado ;

Etiquetas cerradas	Todas las etiquetas deben cerrarse, incluso las que en HTML pueden escribirse sin cierre explícito.	
className	En JSX se utiliza className para asignar clases CSS.	<section className="panel-principal">...</section>
htmlFor	El atributo HTML for se expresa como htmlFor en JSX.	<label htmlFor="correo">Correo</label>
Llaves { }	Las llaves insertan expresiones JavaScript dentro de JSX.	Total: {unidades * precio} €
Fragment	Permite agrupar varios elementos sin añadir un nodo HTML extra.	<><Cabecera /><Contenido /></>
Valores no visibles	false, true, null y undefined no producen texto visible por sí solos al interpolarse en JSX.	{null}
Comentarios JSX	Dentro de JSX los comentarios JavaScript se escriben entre llaves.	{/* comentario de interfaz */}

PELIGRO TÍPICO
JSX se parece a HTML, pero no es una cadena HTML. Las expresiones JavaScript se insertan con { } y los nombres de algunos atributos cambian.

5. Expresiones y renderizado condicional

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Expresión JavaScript	Dentro de { } puede aparecer una expresión que produzca un valor: variables, operaciones, llamadas a funciones o accesos a objetos.	{usuario.nombre.toUpperCase()}
Operador ternario	Permite elegir entre dos fragmentos según una condición.	{activo ? "Servicio activo" : "Servicio detenido"}
Operador &&	Resulta útil para mostrar un elemento sólo cuando una condición es verdadera.	{avisos.length > 0 && Hay avisos pendientes }
Calcular antes del return	Si la lógica necesita varias instrucciones, es preferible resolverla en JavaScript antes del return y usar en JSX el resultado.	const total=precio*cantidad; return {total} ;
Función auxiliar	Una función puede preparar texto o datos que después se muestran en JSX.	const formato=(n)=>`${n.toFixed(1)} h`;

6. Listas, map() y key

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
map() en JSX	Para convertir un array de datos en una colección de elementos React se usa habitualmente map().	{temas.map((tema) => <li key={tema.id}>{tema.nombre})}
key	Cada elemento generado en una lista debe tener una clave estable y única entre sus hermanos para que React identifique correctamente cada elemento.	<li key={tema.id}>...
Evitar índice como key	El índice puede ser aceptable en listas totalmente estáticas, pero una clave propia del dato es preferible si los elementos pueden ordenarse, añadirse o eliminarse.	key={recurso.codigo}
Lista de componentes	map() puede crear componentes personalizados, no sólo etiquetas HTML.	{recursos.map((r) => <Recurso key={r.id} nombre={r.nombre} />)}
Filtrar antes de map	Se pueden encadenar métodos de arrays para	{tareas.filter(t=>t.visible).map(t=><Tarea

	seleccionar datos y después renderizarlos.	key={t.id} {...t} />)}

7. Props: datos de un componente padre a un hijo

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
props	Son los datos que un componente padre proporciona a un componente hijo. El hijo debe tratarlos como valores de sólo lectura.	<Etiqueta nivel={2} texto="Importante" />
Prop string	Un literal de texto puede pasarse entre comillas.	<Cabecera titulo="Unidad 02" />
Prop numérica / booleana	Las expresiones no textuales se pasan entre llaves.	<Progreso valor={75} visible={true} />
Objetos y arrays	También pueden pasarse estructuras completas mediante una expresión JavaScript.	<Resumen datos={estadisticas} />
Acceso mediante props	La función puede recibir un objeto props y consultar sus propiedades.	function Cabecera(props){ return {props.titulo} ; }
Props inmutables	El componente hijo no debe reasignar ni modificar las props recibidas. Si necesita otra representación, crea un valor nuevo.	const tituloLimpio=titulo.trim();

8. Desestructuración, valores por defecto y spread de props

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Desestructurar props	Permite extraer directamente las propiedades necesarias en los parámetros de la función.	function Aviso({titulo, nivel}){ … }
Valor por defecto	Puede asignarse un valor cuando una prop no ha sido proporcionada.	function Boton({texto="Aceptar"}){ return <button>{texto}</button>; }
Renombrar al desestructurar	Una propiedad puede almacenarse en una variable local con otro nombre.	function Ficha({nombre: nombreVisible}){ … }
Spread de props	Un objeto puede expandirse al invocar un componente. Debe usarse cuando el conjunto de propiedades es conocido y coherente.	const datos={titulo:"Tarea",horas:2}; <Ficha {...datos} />
Combinar props	Se pueden pasar propiedades explícitas junto a spread; la posición determina cuál prevalece si se repite una clave.	<Ficha {...datos} horas={3} />

9. children y composición de componentes

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
children	Representa el contenido situado entre la etiqueta de apertura y cierre de un componente.	<Marco> Contenido interno </Marco>
Leer children	Se recibe igual que cualquier otra prop y se inserta en JSX.	function Marco({children}){ return <section>{children}</section>; }
Composición	En lugar de crear componentes gigantes, se combinan piezas especializadas y reutilizables.	<Ficha><CabeceraFicha /><DetalleFicha /></Ficha>
children con varios nodos	children puede contener texto, elementos HTML, componentes o combinaciones de ellos.	<Panel> Estado <Indicador /></Panel>
Etiqueta autocerrada	Si no hay contenido interior, el componente puede escribirse de forma autocerrada.	<Separador />

10. Jerarquía de componentes y flujo de datos

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Padre e hijo	El componente que incluye a otro es su padre en la jerarquía de renderizado. El padre decide qué props entrega al hijo.	<App><Panel /></App>
Flujo descendente	Los datos se transmiten normalmente de padre a hijo mediante props. Esto hace más predecible el origen de cada dato.	<Tarjeta titulo={tituloActual} />
Datos compartidos	Si varios hijos necesitan el mismo dato, suele mantenerse en un ancestro común y se distribuye a cada hijo. El estado se estudiará con detalle en una unidad posterior.	<PanelA usuario={usuario} /><PanelB usuario={usuario} />
Separación de responsabilidades	Un componente puede encargarse de preparar datos y delegar la presentación en componentes más pequeños.	<ListaRecursos recursos={recursosFiltrados} />
Componentes de presentación	Son piezas centradas en mostrar información recibida, ideales para practicar props y composición antes de introducir estado y eventos.	function Dato({etiqueta,valor}){ return {etiqueta}: {valor} ; }

11. CSS, clases y recursos externos

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Importar CSS	En un proyecto Vite puede importarse una hoja CSS desde el módulo/componente para que el empaquetador la procese.	import "./Tarjeta.css";
className	Las reglas CSS se aplican asignando clases mediante className.	<article className="tarjeta-recurso">...</article>
Convención de nombres	Usar nombres de clase específicos reduce colisiones y facilita localizar qué componente utiliza una regla.	.tarjeta-recurso__titulo { font-weight: 700; }
Imagen remota	Una URL puede recibirse mediante props y utilizarse en src. Debe añadirse un texto alt significativo cuando la imagen aporta información.	
Estilo en línea	React permite style con un objeto JavaScript y propiedades camelCase, aunque para el trabajo habitual del curso se priorizarán clases CSS.	<div style={{ backgroundColor: "#eee" }}>...
Recursos locales	Los recursos pueden importarse o ubicarse según la estructura del proyecto. Conviene mantener rutas y nombres consistentes.	import icono from "./assets/icono.svg";

12. Componentes de clase, estado y Hooks: contexto

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Componentes de clase	Son una forma histórica y todavía válida de definir componentes mediante clases que extienden React.Component. En material nuevo se priorizan componentes funcionales.	class Reloj extends React.Component { render(){ return ... ; } }
Estado	Es información interna que puede cambiar durante la vida del componente y provocar un nuevo renderizado. En esta UD sólo se introduce el concepto.	contador = información que cambia al interactuar

Hooks	Funciones especiales de React que permiten usar capacidades como estado y efectos en componentes funcionales. Se estudiarán con detalle en la UT04.	useState(...) // se profundiza en UT04
Props frente a estado	Props: datos recibidos del padre y de sólo lectura. Estado: datos gestionados por el propio componente o por un ancestro.	props.titulo / estadoInterno

LÍMITE DE ESTA UNIDAD
Para realizar la Práctica 2.06 no es necesario dominar Hooks ni eventos. El objetivo aquí es construir componentes estáticos correctamente, componerlos y pasar información con props y children.

13. Depuración y buenas prácticas

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Consola del navegador	Los errores de sintaxis, importación o ejecución aparecen en las herramientas de desarrollo. Lee el primer error antes de corregir los siguientes.	F12 → Consola
React DevTools	La extensión oficial permite inspeccionar el árbol de componentes y las props durante el desarrollo.	Components → seleccionar un componente
Warnings de key	Si se renderiza una lista sin key adecuada, React avisa en desarrollo. El aviso debe corregirse, no ignorarse.	Each child in a list should have a unique "key" prop
No mutar props	Modificar directamente objetos o arrays recibidos dificulta razonar sobre la interfaz. Crea nuevos valores con spread, map o filter cuando sea necesario.	const copia={...datos, activo:true};
Componentes pequeños	Un componente con una responsabilidad clara suele ser más fácil de probar, reutilizar y mantener.	CabeceraFicha / ListaItems / PieFicha
Nombres significativos	Los nombres de componentes describen qué representan; los de props describen qué dato contienen.	<EstadoConexion conectado={true} />
Accesibilidad básica	Usa HTML semántico y texto alternativo en imágenes. La accesibilidad forma parte de una interfaz web de calidad.	

14. Ejemplo integrado original

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Objetivo	Construir una ficha reutilizable de un recurso de aprendizaje con datos recibidos por props, contenido adicional mediante children y una lista de etiquetas generada con map().	Dominio distinto de las prácticas de película.
Componentes	App compone varias TarjetaRecurso. Cada tarjeta recibe título, nivel, url y etiquetas; children contiene la descripción.	App → TarjetaRecurso
Conceptos repasados	Componentes funcionales, props, children, JSX, renderizado condicional, map(), key, className y composición.	Una sola práctica breve integra los conceptos de la UD.

```
// TarjetaRecurso.jsx
import "./TarjetaRecurso.css";

export default function TarjetaRecurso({ titulo, nivel = "Inicial", url, etiquetas = [], children }) {
```

```
  return (
    <article className="tarjeta-recurso">
      <h2>{titulo}</h2>
      <p>Nivel: {nivel}</p>
      {url && <a href={url}>Abrir recurso</a>}
      <div className="tarjeta-recurso__descripcion">{children}</div>
      <ul>
        {etiquetas.map((etiqueta) => (
          <li key={etiqueta}>{etiqueta}</li>
        ))}
      </ul>
    </article>
  );
}
```

```
// App.jsx
import TarjetaRecurso from "./components/TarjetaRecurso.jsx";

const recurso = {
  titulo: "Guía de depuración",
  nivel: "Intermedio",
  url: "https://ejemplo.test/depuracion",
  etiquetas: ["consola", "breakpoints", "errores"]
};

export default function App() {
  return (
    <main>
      <TarjetaRecurso {...recurso}>
        Material para practicar la localización de fallos en aplicaciones de cliente.
      </TarjetaRecurso>
    </main>
  );
}
```

QUÉ DEBES SER CAPAZ DE HACER
Sin copiar este ejemplo, crea otro componente de un dominio distinto, cambia las props, añade al menos un componente hijo y genera una lista a partir de un array. Si puedes hacerlo, estás preparado para abordar una práctica de composición estática como la Práctica 2.06.

15. Resumen comparativo para el examen

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
HTML / componente	Una etiqueta HTML representa un elemento nativo; un componente React es una abstracción reutilizable definida por el programador.	<section>...</section> / <Tarjeta />
JSX / HTML	JSX se parece a HTML, pero admite expresiones JavaScript y tiene reglas propias como className y cierre de etiquetas.	{valor} / className
Props / estado	Props llegan desde el padre y son de sólo lectura. El estado cambia durante la vida del componente y se estudiará en profundidad más adelante.	props.titulo / useState(...)
props / children	Las props nombradas se pasan como atributos; children representa el contenido entre apertura y cierre.	<Caja titulo="A">contenido</Caja>
map / key	map transforma datos en elementos; key identifica de forma estable cada elemento de una lista.	items.map(i=><Fila key={i.id} ... />)
Fragment / div	Ambos agrupan elementos; Fragment no añade un nodo HTML adicional.	<>...</> / ...
className / style	className aplica clases CSS. style recibe un objeto JS y se reserva para casos concretos.	className="aviso" / style={{color:"red"}}

Componente funcional / clase	El material nuevo se centra en funciones. Las clases son relevantes para entender código heredado e historia de React.	function Panel(){} / class Panel extends React.Component
Composición / duplicación	Componer piezas reutilizables evita repetir estructuras y concentra cada responsabilidad en un componente.	<Ficha><Detalle /></Ficha>

ESTRATEGIA DE EXAMEN
Ante un fragmento React identifica: 1) qué componentes intervienen; 2) qué props recibe cada uno; 3) qué expresiones aparecen en JSX; 4) si existe renderizado condicional; 5) si una lista usa map() y una key estable; 6) qué contenido llega mediante children.

Bibliografía y documentación de consulta

Las siguientes referencias se incluyen como documentación de consulta y ampliación. El contenido, la estructura y los ejemplos de este material han sido redactados específicamente para este documento y no reproducen textos, figuras ni ejemplos de las fuentes citadas.

- Documentación oficial de React: https://react.dev/learn
- React Reference. https://react.dev/reference/react
- Vite. Getting Started / Guide. https://vite.dev/guide/